

04 — Project Explanations (simple, speakable, deep enough)

How to explain ANY project — the 5-step formula

1. **Problem:** What business problem existed?
2. **What I built:** The solution in 2–3 lines.
3. **Tech stack and architecture:** Frontend → Backend → Database → Integrations → Deployment.
4. **One challenge:** What was hard and how I solved it.
5. **Result:** Number or impact.

Speak for 1–2 minutes first. Then let the interviewer ask deeper questions.

Before each interview, revise the 3 resume projects (TorahAnytime, Hospital CRM, BikeShopShippers) and the WhatsApp platform. Those are the most likely questions.

PROJECT 1 — Hospital Appointment CRM (Atiya Healthcare) ★ main project

Stack: Node.js, Express 5, React 19, Redux Toolkit, SQL Server (2 databases), Redis, Joi, Winston, Swagger, IIS + iisnode

Spoken explanation (1.5 min)

"At Atiya Healthcare, call-centre agents book clinic appointments for patients. They used an old ASP.NET WebForms CRM that was slow and hard to change.

I rebuilt it as a Node.js + Express backend and a React frontend, on top of the same SQL Server databases, because many other systems depended on those databases.

The backend has 47 API modules — appointments, patients, call dispositions, complaint tickets, consultation forms and courier tracking. It follows a layered structure: routes, Joi validation, controller, service and repository with raw parameterised SQL.

It integrates with the Ameyo dialer — the CRM opens inside the dialer as an iframe and the backend can auto-dispose calls. On booking, it sends a WhatsApp confirmation through Exotel and an SMS through ValueFirst.

The biggest challenge was size — 460+ tables across two databases. I wrote a script that read the SQL schema and generated 463 model files automatically, which saved weeks.

I deployed it on Windows IIS with iisnode, running 4 Node processes, with health endpoints and log rotation. Agents use it every day, and we reduced processing time by around 30%."

Architecture (draw this if asked)

```
Ameyo Dialer → iframe → React SPA (IIS static site)
    ↓ axios (agent headers / JWT for admin)
    Express 5 API on IIS + iisnode (4 processes)
    routes → Joi validation → controller → service → repository (raw SQL)
    ↓
    ↓
SQL Server: CRM DB + Clinic DB    Ameyo API / Exotel WhatsApp / SMS / Courier APIs
Redis: dialer session state
```

Deep-dive Q&A

- **Why raw SQL instead of an ORM?** "The legacy schema had 460+ tables and complex queries. Raw parameterised SQL gave full control over performance. Parameters prevent SQL injection."
- **How did you connect to two databases?** "One Database class with two named connection pools — one for the CRM DB and one for the clinic DB — with a concurrency limit."
- **How does login work inside the dialer?** "The dialer already authenticates the agent. It passes user and session IDs by URL or postMessage. The frontend sends them as headers, and the backend resolves the agent and caches it for 5 minutes. Admin users log in with JWT."
- **What if WhatsApp sending fails?** "It never breaks the booking. I retry only on network errors, 5xx or 429, up to 3 times with exponential backoff. Failures are logged in a table. Delivery status comes back through a webhook protected with a secret token."
- **How did you handle the old ASP.NET passwords?** "I re-implemented the same encryption in Node (PBKDF2 + AES-256-CBC) so existing users didn't need to reset passwords."
- **RBAC?** "Page-level rights from the database — view, download, edit. Middleware checks them on each route."
- **Processing time reduced 30% — how?** "Optimised SQL queries and indexes, fewer round trips per screen, caching of lookups, and connection pooling."

PROJECT 2 — WhatsApp Automation Platform (Atiya Healthcare)

Stack: Node.js, Express, Socket.IO, MongoDB Atlas (Mongoose), React, Exotel WhatsApp Business API, Cloudinary, PM2, Apache on Cloudways

Spoken explanation

"The company talked to customers on two WhatsApp Business numbers, but agents used phones, so there was no shared inbox, history or reports.

I built a real-time WhatsApp inbox. Exotel sends incoming messages and delivery receipts to my Express webhook. I save them in MongoDB and push them instantly to agents' browsers using Socket.IO. Agents reply from the web UI, and the backend automatically sends the reply from the same business number the customer messaged.

It has unread counts, media upload through Cloudinary, campaign/template messages, and admin reports using MongoDB aggregation.

One challenge: a delivery status sometimes arrived before the outgoing message was saved. I fixed it by upserting by the Exotel message ID, so the order doesn't matter. This also makes webhook retries safe — no duplicates.

I deployed it on Cloudways with PM2 and an Apache reverse proxy that also forwards WebSocket traffic."

Deep-dive Q&A - Why MongoDB here? "Messages are document-shaped with different payloads, high write rate and simple access patterns by contact and time." - **Indexes?** "Compound indexes like `{to, isRead}` and `{from, to, isRead}` for unread counts, and `{direction, timestamp}` for lists." - **How will Socket.IO scale to many servers?** "Use the Socket.IO Redis adapter so events reach all instances, plus sticky sessions on the load balancer." - **Webhook security?** "Webhooks are public by nature. API routes use an API key. The improvement is signature verification on webhooks." - **Voice notes rejected by WhatsApp?** "I transcoded audio to MP3 through Cloudinary, which WhatsApp accepts."

PROJECT 3 — TorahAnytime (Morpheme Webnexus, Israeli client) ★ resume project

Stack (resume): Node.js, MySQL, Redis, AWS, Docker. *Also used in the project per your notes:* RabbitMQ, tRPC, DigitalOcean, ClickHouse, RethinkDB, Kubernetes.

What it is: A global Torah lecture (religious media) and donation platform. Speakers register, upload lectures and classes, and run donation campaigns. It serves many concurrent users worldwide.

Spoken explanation

"TorahAnytime is a high-traffic media and donation platform for an Israeli client. Speakers upload lectures, users watch and listen, create playlists, and donate to campaigns.

I was the lead developer from our side. I assigned tickets to developers, reviewed and merged pull requests, and deployed releases to production.

I also built features myself — a like system, editable playlists, legacy speakers pages, and a most-viewed filter. On the backend I optimised REST APIs and background processing so the platform stayed fast under high traffic.

I worked with Docker for containers, Redis for caching, MySQL as the main database, and queues for background jobs. This project grew my DevOps skills a lot."

How to explain the features technically (*say only what matches what you really built*) - **Like button:** "A likes table with a unique key on user + lecture so a user can like only once. The API toggles like/unlike. The like count is stored on the lecture and updated in the same transaction, so we don't count rows on every page load." - **Editable playlists:** "Playlist and playlist-items tables with a position column. Reordering updates positions in one transaction. Only the owner can edit — checked in middleware." - **Most-viewed filter:** "View counts are recorded in the background, not in the request path. The top lists are cached in Redis with a TTL, so the database isn't hit on every request." - **Background processing:** "Heavy work like view-count updates, notifications and media processing goes to a queue (RabbitMQ) and is processed by workers, so the API responds quickly."

Deep-dive Q&A - What did your PR review check? "Correct logic, edge cases, SQL performance (indexes, N+1 queries), security like input validation and auth checks, naming, and that tests or manual test steps exist." - **How did you deploy?** "Code merged to the main branch, Docker image built, deployed to the server, then smoke-tested key pages. If something broke, we rolled back to the previous image." - **How did you handle high traffic?** "Caching hot data in Redis, indexes on frequent filters, pagination, moving heavy work to background workers, and horizontal scaling of containers." - **How did you work with the client?** "Calls and Slack/Jira with the client team in Israel, understanding tickets, giving estimates, and demoing features."

PROJECT 4 — BikeShopShippers (Morpheme Webnexus) ★ resume project

Stack: Medusa.js (Node.js commerce framework), PostgreSQL, Redis, Stripe API, UPS API, Google Places API, Next.js/React, GitHub Actions CI/CD

What it is: A logistics platform for shipping heavy items like bikes and vehicles. The platform makes bulk deals with shipping companies for discounts. Shop owners register and ship through it at lower rates.

Spoken explanation

"BikeShopShippers is a logistics platform. Bike and vehicle shop owners need to ship heavy items to customers, which is expensive. The platform negotiates bulk discounts with carriers, and shop owners book shipments through it.

I built it from scratch on Medusa.js, which is a Node.js headless commerce framework, with PostgreSQL. I integrated UPS APIs for rates, packaging, shipping labels and tracking, Stripe for payments, and Google APIs for address autocomplete. I also built the shipment dashboard UI.

I set up CI/CD with GitHub Actions and used GitHub Projects to plan and assign tasks, and I led the releases.

One challenge was keeping payment and shipment in sync. I used Stripe webhooks — the label is created only after Stripe confirms payment, and webhook handling is idempotent so a retry doesn't create two labels."

Deep-dive Q&A - Why Medusa.js? "It gives ready commerce modules — orders, customers, payments, fulfilment providers — and it's extendable in Node. I added a custom UPS fulfilment provider instead of building commerce from zero." - **How does Stripe payment work?** "Backend creates a PaymentIntent, frontend confirms the card with Stripe.js, and Stripe sends a webhook. I verify the webhook signature and then mark the order paid." - **UPS integration?** "OAuth token from UPS, then Rating API for prices, Shipping API for labels, and Tracking API for status updates." - **What did the CI/CD pipeline do?** "On push: install, lint, test and build. On merge to main: deploy to the server automatically."

PROJECT 5 — ViewReports BI platform (Atiya) — great backend story

Stack: Express 5, SQL Server, PostgreSQL, SQLite read-model, Redis, BullMQ, node-cron, JWT + email OTP, ExcelJS, React

"Management needed 45+ call-centre reports — agent performance, campaign conversion, transfers, revenue. Source tables had 50 million+ rows, so live queries were too slow.

I built an ETL sync that copies data in chunks using keyset pagination into a local SQLite read-model. A BullMQ worker runs the sync on a schedule. Reports read from the read-model with caching that is invalidated when new data syncs.

Login uses JWT plus an email OTP, and every report has a permission key for RBAC. Excel export has a row-limit guard (around 800k rows) that returns HTTP 413 instead of crashing the server.

Challenge: one heavy report on synchronous SQLite blocked the Node event loop and froze the app. I added a date-range guard, moved index building to a worker thread and separated the sync worker. After that there were no freezes."

PROJECT 6 — QMS call-quality report (Atiya) — query optimisation story

"QA auditors needed all details of one call — both call legs, agent, disposition and the recording. The original SQL Server view used a per-row OUTER APPLY and took about 92 seconds.

I bypassed the view, queried base tables with UNION ALL, and added the right indexes. It came down to 200–350 milliseconds. I also built an audio streaming proxy that supports HTTP Range requests so recordings play and seek inside the page."

PROJECT 7 — CallDropAutoDial (Atiya) — automation story

"When customer calls dropped or were abandoned, nobody called back. I built a Node service with node-cron that runs every 30 minutes, finds dropped calls in PostgreSQL using DISTINCT ON and NOT EXISTS, and uploads them to dialer campaigns for automatic call-back.

Safety: a DRY_RUN flag to test without dialling, a re-entrancy guard so two runs don't overlap, and only one process on IIS so the cron never fires twice.

Interesting bug: results were always empty. The table I read was filled by an hourly ETL job, so the 30-minute window was always empty. I switched to the live parent table."

PROJECT 8 — Patient-CRM (Atiya) — caching story

"Agents search a phone number and see the patient's full 30-day call history across all their numbers. The first version took minutes. I added a 3-tier cache: in-memory LRU, then Redis, then a SQLite mirror, and finally PostgreSQL. Lookups dropped to milliseconds. If Redis is down, the code falls through to the next tier — slower, but not broken."

Other projects (short answers if asked)

Project	Company	2-line answer
PERK	Morpheme	Event management platform. I built serverless CRUD APIs on AWS Lambda with TypeScript, a PDF-to-image converter and a PDF editor for event layout maps.
FamilyOne	Morpheme	Family discount network. I optimised Lambda functions, wrote serverless YAML configs, load-tested APIs with JMeter and set up Sentry and Codecov.
Nosotros LMS	Morpheme	LMS for Spanish schools with admin, school, teacher, parent and student portals. I set up Bitbucket CI/CD with Sentry, Codecov, Jira and Slack.
ShowTrail	Morpheme	Coupon promotion platform sharing FamilyOne's admin. I trained Intercom AI workflows for customer support.
SmartBiz	Morpheme	B2B onboarding with dynamic forms per sector. I built the APIs and UI.

Project	Company	2-line answer
Astrology platform	MSAI	Built from scratch: tech stack, Prokerala astrology APIs, chat + audio/video consultation.
ThePNAC.com	MSAI	Built end to end and deployed with PM2 + NGINX on the company server.
Real Estate	Personal	MERN + Firebase: listings, filters, buyer/seller dashboards. Deployed on Render.
E-commerce	Personal	MERN: auth, products, cart, orders, payments, admin dashboard.

Common project questions (ready answers)

"What was the most challenging project?" → ViewReports (scale + event loop freeze) or QMS (92 s → 300 ms).

"Did you build it alone?"

"At Atiya, yes — I'm the main developer. I worked with business users for requirements and testing, and the IT team for servers and database access. At Morpheme I worked in a team and led TorahAnytime."

"What would you improve in your project?"

"Add TypeScript, more unit tests per service, move secrets to a vault, add a CI/CD pipeline for the IIS apps, and add the Redis adapter for Socket.IO scaling."

"How many users?" → Give honest approximate numbers: "Used daily by the call-centre agents and managers" (confirm the real count — e.g., 100+ agents).